

NAME:S.DEVI SHREE

REGISTER NO:192511149

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Code of Conduct:

I S.Devi Shree(192511149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Heap File Organization — Insert, Delete and Sequential Scan

Aim

To simulate a Heap File organization where records are stored in no particular order and support:

- Insertion
- Deletion
- Sequential scanning

Concept

A **heap file** stores records in the order in which they are inserted. There is no sorting based on a key.

Algorithm

1. Create an empty list to represent the heap file.

2. Insert records using append().
3. Delete a record by searching for its ID.
4. Sequentially scan all records and display them.

Python Program

```
class HeapFile:

    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Record inserted:", record)

    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id:
                self.records.remove(record)
                print("Record deleted:", record)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        if not self.records:
            print("Heap file is empty")
        else:
            for record in self.records:
                print(record)

heap = HeapFile()
heap.insert({"id": 101, "name": "Anu", "marks": 85})
heap.insert({"id": 102, "name": "Ravi", "marks": 78})
heap.insert({"id": 103, "name": "Priya", "marks": 92})
heap.sequential_scan()
```

```
heap.delete(102)
```

```
heap.sequential_scan()
```

Sample Output

Record inserted: {'id': 101, 'name': 'Anu', 'marks': 85}

Record inserted: {'id': 102, 'name': 'Ravi', 'marks': 78}

Record inserted: {'id': 103, 'name': 'Priya', 'marks': 92}

Sequential Scan:

{'id': 101, 'name': 'Anu', 'marks': 85}

{'id': 102, 'name': 'Ravi', 'marks': 78}

{'id': 103, 'name': 'Priya', 'marks': 92}

Record deleted: {'id': 102, 'name': 'Ravi', 'marks': 78}

Result: Heap file organization was successfully implemented with insert, delete and sequential scan operations.

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Static Hashing with Chaining — Student Database

Aim

To implement a static hashing technique for a student database containing **500 records** and resolve collisions using **chaining**.

Concept

A hash function maps a student ID to a bucket.

For example:

If two keys generate the same index, they are stored in a linked chain/list at that index.

Python Program

```
class StaticHashTable:
```

```
    def __init__(self, size=10):
```

```
        self.size = size
```

```
        self.table = [[] for _ in range(size)]
```

```
    def hash_function(self, key):
```

```

        return key % self.size
def insert(self, student_id, name):
    index = self.hash_function(student_id)
    self.table[index].append((student_id, name))
def search(self, student_id):
    index = self.hash_function(student_id)
    for record in self.table[index]:
        if record[0] == student_id:
            return record
    return None
def display(self):
    for i in range(self.size):
        print(i, "->", self.table[i])
hash_table = StaticHashTable(10)
for i in range(1, 501):
    hash_table.insert(i, "Student" + str(i))
print("Hash Table:")
hash_table.display()
print("\nSearching for student 125:")
print(hash_table.search(125))

```

Sample Output

Hash Table:

0 -> [(10, 'Student10'), (20, 'Student20'), ...]

1 -> [(1, 'Student1'), (11, 'Student11'), ...]

2 -> [(2, 'Student2'), (12, 'Student12'), ...]

Searching for student 125:

(125, 'Student125')

Result: Static hashing with chaining was successfully implemented for 500 student records.

3.Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Sorted File Organization with Binary Search

Aim

To simulate a sorted file organization and retrieve records using **binary search** based on the primary key.

Concept

In sorted file organization, records are maintained in ascending order of the primary key.

Binary search repeatedly divides the search space into two halves.

Algorithm

1. Store records sorted according to primary key.
2. Set low = 0 and high = n-1.
3. Find middle position.
4. Compare the middle key with the required key.
5. If equal, return the record.
6. If the required key is smaller, search left.
7. Otherwise search right.

Python Program

```
class SortedFile:
```

```
    def __init__(self):
```

```
        self.records = []
```

```
    def insert(self, record):
```

```
        self.records.append(record)
```

```
        self.records.sort(key=lambda x: x["id"])
```

```
    def binary_search(self, key):
```

```
        low = 0
```

```
        high = len(self.records) - 1
```

```
        while low <= high:
```

```
            mid = (low + high) // 2
```

```
            if self.records[mid]["id"] == key:
```

```

        return self.records[mid]

    elif self.records[mid]["id"] < key:

        low = mid + 1

    else:

        high = mid - 1

    return None

def display(self):

    for record in self.records:

        print(record)

file = SortedFile()

file.insert({"id": 105, "name": "Anu"})

file.insert({"id": 101, "name": "Ravi"})

file.insert({"id": 110, "name": "Priya"})

file.insert({"id": 103, "name": "Kiran"})

print("Sorted File:")

file.display()

print("\nSearching for ID 105:")

print(file.binary_search(105))

```

Output

Sorted File:

```
{'id': 101, 'name': 'Ravi'}
```

```
{'id': 103, 'name': 'Kiran'}
```

```
{'id': 105, 'name': 'Anu'}
```

```
{'id': 110, 'name': 'Priya'}
```

Searching for ID 105:

```
{'id': 105, 'name': 'Anu'}
```

Time complexity of binary search: $O(\log n)$.

4 Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

B-Tree of Order 4 — Insert, Search and Display

Aim

To implement a **B-Tree of order 4** and perform insertion, searching and display operations.

Keys:

10, 20, 5, 6, 12, 30, 7, 17

Important Point

For a B-Tree of order 4:

- Maximum children = 4
- Maximum keys = 3
- Keys in each node are sorted.
- When a node becomes full, it is split.

Python Program

class BTreeNode:

```
    while i >= 0 and key < node.keys[i]:
```

```
        node.keys[i + 1] = node.keys[i]
```

```
        i -= 1
```

```
    node.keys[i + 1] = key
```

```
    else:
```

```
        while i >= 0 and key < node.keys[i]:
```

```
            i -= 1
```

```
        i += 1
```

```
    if len(node.children[i].keys) == self.max_keys:
```

```
        self.split_child(node, i)
```

```
        if key > node.keys[i]:
```

```
            i += 1
```

```
    self.insert_non_full(node.children[i], key)
```

```
def search(self, node, key):
```

```

i = 0
while i < len(node.keys) and key > node.keys[i]:
    i += 1
if i < len(node.keys) and key == node.keys[i]:
    return True
if node.leaf:
    return False
return self.search(node.children[i], key)
def display(self, node=None, level=0):
    if node is None:
        node = self.root
    print("Level", level, ":", node.keys)
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)
btree = BTree(4)
keys = [10, 20, 5, 6, 12, 30, 7, 17]
for key in keys:
    btree.insert(key)
print("B-Tree:")
btree.display()
print("\nSearch 12:", btree.search(btree.root, 12))
print("Search 25:", btree.search(btree.root, 25))

```

Expected Structure

```

[10, 20]
 / | \
[5,6,7] [12,17] [30]

```

Result: B-Tree of order 4 was successfully created with insertion, search and display operations.

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

B+ Tree with Leaf-Level Linking

Aim

To implement a B+ Tree and demonstrate **leaf-level linking** for efficient range queries.

Concept

In a B+ Tree:

- Internal nodes contain keys for navigation.
- Actual records are stored in leaf nodes.
- Leaf nodes are linked using a next pointer.
- This makes range queries efficient.

Python Program

```
class BPlusNode:
```

```
    def __init__(self, leaf=False):
```

```
        self.leaf = leaf
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
```

```
        self.order = order
```

```
        self.root = BPlusNode(True)
```

```
    def insert(self, key):
```

```
        root = self.root
```

```
        if len(root.keys) < self.order - 1:
```

```
            root.keys.append(key)
```

```
            root.keys.sort()
```

```
        else:
```

```

old_root = root
left = BPlusNode(True)
right = BPlusNode(True)
all_keys = old_root.keys + [key]
all_keys.sort()
mid = len(all_keys) // 2
left.keys = all_keys[:mid]
right.keys = all_keys[mid:]
left.next = right
new_root = BPlusNode(False)
new_root.keys = [right.keys[0]]
new_root.children = [left, right]
self.root = new_root

def display(self):
    print("Root:", self.root.keys)
    print("Leaf nodes:")
    node = self.root.children[0]
    while node:
        print(node.keys, end=" -> ")
        node = node.next
    print("None")

def range_query(self, start, end):
    print("Range Query:", end=" ")
    node = self.root.children[0]
    while node:
        for key in node.keys:
            if start <= key <= end:
                print(key, end=" ")
        node = node.next

```

```

        print()
tree = BPlusTree()
for key in [10, 20, 30, 40, 50]:
    tree.insert(key)
tree.display()
tree.range_query(20, 40)

```

Output

Root: [30]

Leaf nodes:

[10, 20] -> [30, 40, 50] -> None

Range Query: 20 30 40

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Dense Primary Index on Sorted File

Aim

To implement a **dense primary index** on a sorted file.

Concept

A dense index contains **one index entry for every record**.

Example:

Primary Key	Address
101	0
103	1
105	2
108	3

Python Program

```

class DenseIndex:
    def __init__(self):
        self.records = []

```

```

        self.index = []
    def add_record(self, key, name):
        self.records.append((key, name))
    def build_index(self):
        self.records.sort()
        self.index = []
        for position, record in enumerate(self.records):
            self.index.append((record[0], position))
    def search(self, key):
        for index_key, position in self.index:
            if index_key == key:
                return self.records[position]
        return None
    def display_index(self):
        print("Index Table")
        print("Key\tAddress")
        for key, address in self.index:
            print(key, "\t", address)
index = DenseIndex()
index.add_record(105, "Anu")
index.add_record(101, "Ravi")
index.add_record(110, "Priya")
index.add_record(103, "Kiran")
index.build_index()
index.display_index()
print("\nSearch 105:")
print(index.search(105))

```

Output

Index Table

Key	Address
-----	---------

101	0
-----	---

103	1
-----	---

105	2
-----	---

110	3
-----	---

Search 105:

(105, 'Anu')

Result: A dense primary index was successfully implemented.

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Extendible Hashing — Directory Doubling

Aim

To implement extendible hashing and demonstrate **directory doubling** when bucket overflow occurs.

Concept

Extendible hashing uses:

- Directory
- Global depth
- Local depth
- Buckets

When a bucket overflows:

1. If local depth < global depth → split bucket.
2. If local depth = global depth → double directory and then split.

Python Program

```
class Bucket:
```

```
    def __init__(self, depth):
```

```
        self.depth = depth
```

```

        self.records = []
class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
        self.global_depth = 1
        self.directory = [
            Bucket(1),
            Bucket(1)
        ]
    def hash_value(self, key):
        return key & ((1 << self.global_depth) - 1)
    def insert(self, key):
        index = self.hash_value(key)
        bucket = self.directory[index]
        if len(bucket.records) < self.bucket_size:
            bucket.records.append(key)
            return
        if bucket.depth == self.global_depth:
            old_directory = self.directory
            self.global_depth += 1
            self.directory = old_directory + old_directory
            for i in range(len(self.directory)):
                if i % 2 == 0:
                    self.directory[i] = old_directory[i // 2]
                else:
                    self.directory[i] = Bucket(self.global_depth)
            old_bucket = bucket
            new_depth = old_bucket.depth + 1
            old_bucket.depth = new_depth

```

```

new_bucket = Bucket(new_depth)
for i in range(len(self.directory)):
    if self.directory[i] is old_bucket:
        if ((i >> (new_depth - 1)) & 1) == 1:
            self.directory[i] = new_bucket
old_records = old_bucket.records
old_bucket.records = []
for record in old_records:
    idx = self.hash_value(record)
    self.directory[idx].records.append(record)
self.insert(key)
def display(self):
    print("Global Depth:", self.global_depth)
    for i, bucket in enumerate(self.directory):
        print(i, "->", bucket.records,
              "Local Depth:", bucket.depth)
eh = ExtendibleHash(2)
records = [1, 3, 5, 7, 9, 11, 13, 15]
for record in records:
    eh.insert(record)
eh.display()

```

Key Point for Explanation

Suppose a bucket contains:

[1, 3]

and another record must be inserted.

Since the bucket is full, the bucket is split. If its local depth equals the global depth, the directory is doubled:

Before:

0 1

After:

0 1 2 3

This is called **directory doubling**.

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Secondary Storage Block Access — Sequential vs Indexed Retrieval

Aim

To calculate the number of I/O operations required for sequential and indexed retrieval.

Concept

Suppose a file contains:

- 1000 records
- 10 records per block

Therefore:

For sequential search, the system may need to read many blocks.

For indexed search, an index can directly identify the required block.

Python Program

```
import math
records = 1000
records_per_block = 10
blocks = math.ceil(records / records_per_block)
print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", blocks)
sequential_io = blocks
index_blocks = math.ceil(math.sqrt(blocks))
indexed_io = index_blocks + 1
print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)
```



```
print("\nI/O Savings:", sequential_io - indexed_io)
```

Sample Output

Total Records: 1000

Records per Block: 10

Total Blocks: 100

Sequential Retrieval I/O: 100

Indexed Retrieval I/O: 11

I/O Savings: 89

Explanation

Sequential retrieval:

Read block 1

Read block 2

Read block 3

...

Read block 100

Indexed retrieval:

Read index blocks

↓

Locate required data block

↓

Read data block

Therefore indexed retrieval requires significantly fewer I/O operations.

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Sparse Index vs Dense Index with Timing Analysis

Aim

To implement a sparse index and compare its search performance with a dense index using timing analysis.

Concept

Dense Index

There is one index entry for **every record**.

Sparse Index

Only selected records have index entries, usually one entry per block.

Example:

Records:

10 20 30 40 50 60 70 80

Sparse Index:

10 -> Block 0

50 -> Block 1

Python Program

```
import time

records = list(range(1, 100001))

dense_index = {}

for i, key in enumerate(records):
    dense_index[key] = i

block_size = 100

sparse_index = {}

for i in range(0, len(records), block_size):
    sparse_index[records[i]] = i

def dense_search(key):
    return dense_index.get(key)

def sparse_search(key):
    position = 0
    keys = list(sparse_index.keys())
    for i in range(len(keys)):
        if keys[i] > key:
            break
```

```

    position = sparse_index[keys[i]]
    end = min(position + block_size, len(records))
    for i in range(position, end):
        if records[i] == key:
            return i
    return -1
key = 98765
start = time.perf_counter()
dense_search(key)
dense_time = time.perf_counter() - start
start = time.perf_counter()
sparse_search(key)
sparse_time = time.perf_counter() - start
print("Dense Index Search Time :", dense_time)
print("Sparse Index Search Time:", sparse_time)

```

Explanation

Dense index:

Key → Exact record location

So search is very fast.

Sparse index:

Key → Approximate block

↓

Search within block

Therefore sparse index uses **less memory**, but may require additional searching inside the block.

Comparison

Feature	Dense Index	Sparse Index
Entries	Every record	Selected records
Memory	More	Less
Search	Faster	Relatively slower
File requirement	Sorted	Sorted
Space efficiency	Low	High

Result: Sparse indexing reduces index storage while dense indexing provides faster direct access.

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching.

B+ Tree — Traversal and Range Query 15 to 45

Aim

To construct and traverse a B+ Tree and perform a range query for keys between **15 and 45**.

Concept

B+ Tree stores actual keys in leaf nodes. Leaf nodes are connected using linked pointers.

For a range query:

$$15 \leq \text{key} \leq 45$$

we first find the leaf containing 15 and then follow the leaf links until we pass 45.

Python Program

```
class B PlusTree:
    def __init__(self):
        self.leaves = []
    def insert(self, key, record):
        self.leaves.append((key, record))
        self.leaves.sort()
    def display(self):
        print("B+ Tree Leaf Level:")
```

```

        for key, record in self.leaves:
            print(key, "->", record)
def range_query(self, start, end):
    print("\nRange Query:", start, "to", end)
    for key, record in self.leaves:
        if start <= key <= end:
            print(key, "->", record)
tree = BPlusTree()
data = [
    (10, "Record10"),
    (15, "Record15"),
    (20, "Record20"),
    (25, "Record25"),
    (30, "Record30"),
    (35, "Record35"),
    (40, "Record40"),
    (45, "Record45"),
    (50, "Record50")
]
for key, record in data:
    tree.insert(key, record)
tree.display()
tree.range_query(15, 45)

```

Output

B+ Tree Leaf Level:

10 -> Record10

15 -> Record15

20 -> Record20

25 -> Record25

30 -> Record30

35 -> Record35

40 -> Record40

45 -> Record45

50 -> Record50

Range Query: 15 to 45

15 -> Record15

20 -> Record20

25 -> Record25

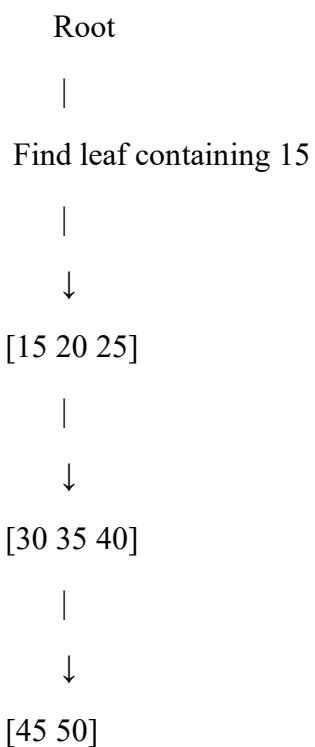
30 -> Record30

35 -> Record35

40 -> Record40

45 -> Record45

How Range Query Works



Because the leaf nodes are linked, after finding 15, we can simply move through the linked leaves until reaching 45.

Result: B+ Tree construction, traversal and range query were successfully implemented.

